

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Doğal Dil İşleme dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (Hugging Face LLM Course, Transformers, Datasets, Tokenizers kütüphaneleri) kendi orijinal lisanslarına (Apache 2.0, MIT, CC-BY 4.0) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.

Using 🤗 Transformers — Cheat Sheet

1. 🤗 Transformers nedir?

Transformers, metin gibi veriler üzerinde önceden eğitilmiş modelleri (*pretrained models*) kullanmamızı sağlayan bir kütüphanedir. Hugging Face, bu modelleri yüklemeyi ve farklı doğal dil işleme (*Natural Language Processing — NLP*) görevlerinde kullanmayı kolaylaştırır.

Bir modeli kullanırken genellikle üç adım vardır:

1. **Preprocessing (ön işleme):** Girdi metni modelin anlayacağı biçime dönüştürme.
2. **Inference (çıkartım):** Modelin girdiyi işleyip sonuç üretmesi.
3. **Postprocessing (son işlem):** Model çıktısını okunabilir bir sonuca dönüştürme.

Pipeline nasıl çalışır?

pipeline, bu adımları tek bir arayüz üzerinden çalıştırmamızı sağlar. Başlangıç seviyesinde modelin ve tokenizer'ın ayrıntılarıyla tek tek uğraşmadan sonuç alabiliriz.

```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")
result = classifier("I love learning about language models!")
print(result)
```

Kod ne yapıyor?

- pipeline: Transformers kütüphanesinden pipeline aracını içe aktarır.
- "sentiment-analysis": Yapılacak görevin duygu analizi olduğunu belirtir.
- classifier(...): Verilen cümleyi analiz eder.
- print(result): Sonucu ekrana yazdırır. Çıktıda genellikle bir etiket (*label*) ve güven skoru (*score*) bulunur.

Kısaca: Pipeline, metni hazırlar, modele gönderir ve sonucu düzenleyerek verir.

2. Model ve Tokenizer

Modeli doğrudan metinle çalıştırmak çoğu zaman mümkün değildir. Metnin önce sayısal temsile dönüştürülmesi gerekir. Bu noktada **tokenizer** ve **model** birlikte çalışır.

- **Tokenizer (belirteçleyici):** Metni token adı verilen parçalara ayırır ve bunları sayısal kimliklere (*token IDs*) dönüştürür.
- **Model:** Bu sayısal girdileri işler ve görevine uygun çıktılar üretir.
- **Checkpoint (model kaydı):** Modelin ağırlıklarını ve ilgili yapılandırma dosyalarını içeren kayıt.
- **Pretrained model (önceden eğitilmiş model):** Daha önce büyük miktarda veriyle eğitilmiş ve yeniden kullanılabilen model.

Tokenizer'ı yüklemek

```
from transformers import AutoTokenizer
```

```
checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"  
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

```
inputs = tokenizer("I love learning about NLP!")  
print(inputs)
```

Kod ne yapıyor?

- **AutoTokenizer:** Verilen checkpoint'e uygun tokenizer sınıfını seçer.
- **from_pretrained(checkpoint):** İlgili tokenizer ayarlarını indirir veya kayıtlı sürümünü yükler.
- **tokenizer("..."):** Metni token ID'leri gibi modelin kullanacağı girdilere dönüştürür.
- **inputs:** Genellikle input_ids ve modele göre attention_mask gibi alanlar içeren bir sözlüktür.

Tokenizer ile ilgili temel terimler

- **Token:** Metnin model tarafından işlenen parçası. Bir token kelimenin tamamı, kelimenin bir bölümü veya noktalama işareti olabilir.
- **Token ID:** Token'ın sözlükteki sayısal karşılığı.
- **Special tokens (özel token'lar):** Modelin girdiyi anlamlandırmasına yardımcı olan özel işaretler.
- **Vocabulary (kelime/token sözlüğü):** Token'larla bunların ID'leri arasındaki eşleştirme.

Tokenizer'ın kullandığı parçalama yöntemi modele göre değişebilir. Yaygın yaklaşımlar arasında **WordPiece**, **BPE (Byte-Pair**

Encoding) ve Unigram bulunur.

Faydalı tokenizer yöntemleri

```
tokens = tokenizer.tokenize("I love learning about NLP!")
ids = tokenizer.convert_tokens_to_ids(tokens)
text = tokenizer.decode(ids)
```

```
print(tokens)
print(ids)
print(text)
```

- `tokenize()`: Metni token parçalarına ayırır.
- `convert_tokens_to_ids()`: Token parçalarını sayısal ID'lere çevirir.
- `decode()`: ID'leri yeniden okunabilir metne dönüştürür.

Not: Decode edilen metin, özel token'lar ve tokenizer'ın kuralları nedeniyle ilk yazılan metinle her zaman karakter karakter aynı olmayabilir.

3. Model türleri: AutoModel ve görev başlıkları

Transformers'ta farklı ihtiyaçlar için farklı model sınıfları vardır.

- **AutoModel:** Temel Transformer modelini yükler. Genellikle gizli durumlar (*hidden states*) gibi ara temsiller üretir.
- **AutoModelForSequenceClassification:** Metin sınıflandırma görevleri için sınıflandırma başlığı (*classification head*) eklenmiş modeli yükler.
- **Classification head (sınıflandırma başlığı):** Temel modelin temsilini sınıf tahmini için kullanılabilecek çıktıya dönüştüren katman.

Duygu analizi gibi bir sınıflandırma görevi için görev odaklı sınıf kullanılır:

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
```

```
checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
```

```
inputs = tokenizer("I love learning about NLP!", return_tensors="pt")
outputs = model(**inputs)
```

```
print(outputs.logits)
```

Kod ne yapıyor?

- Tokenizer ve sınıflandırma modeli aynı checkpoint üzerinden yüklenir.
- `return_tensors="pt"`: Girdileri PyTorch tensörleri biçiminde döndürür. (pt, PyTorch anlamına gelir.)
- `model(**inputs)`: Sözlükteki girdileri modele ayrı argümanlar olarak iletir.
- `outputs.logits`: Sınıflandırma sınıfları için modelin ürettiği ham skorları içerir.

logits ve olasılık

Logits, modelin doğrudan ürettiği ham sınıf skorlarıdır; tek başına olasılık olarak yorumlanmamalıdır. **Softmax**, bu skorları toplamı 1 olan olasılık değerlerine dönüştürmek için kullanılabilir.

```
import torch
```

```
probabilities = torch.softmax(outputs.logits, dim=-1)
```

```
print(probabilities)
```

Burada `dim=-1`, işlemin son boyut üzerinde yapılacağını belirtir. Sınıflandırmada bu boyut genellikle sınıf skorlarını içerir.

Önemli ayırım: `AutoModel` ile `AutoModelForSequenceClassification` aynı şey değildir. Birincisi temel temsilleri, ikincisi ise sınıflandırma görevine yönelik çıktıları verir.

4. Hidden states (gizli durumlar) ve çıktı boyutları

Temel Transformer modeli, girdideki token'lar için sayısal temsiller üretir. Bunlara **hidden states (gizli durumlar)** denir.

Bir modelin `last_hidden_state` çıktısının boyutları genellikle şu sıradadır:

[batch_size, sequence_length, hidden_size]

- **batch_size (toplu iş boyutu):** Aynı anda işlenen örnek sayısı.
- **sequence_length (dizi uzunluğu):** Girdideki token sayısı.
- **hidden_size (gizli katman boyutu):** Her token için üretilen temsilin sayısal uzunluğu.

Örneğin şekil [2, 10, 768] olsaydı, bu iki örnek, her örnekte 10 token ve her token için 768 boyutlu temsil olduğu anlamına gelirdi.

Bu yalnızca boyutları açıklayan bir örnektir; her modelde bu sayılar aynı değildir.

5. Birden fazla metin: batching, padding ve truncation

Batching (toplu işleme), birden fazla girdiyi tek seferde modele vermektir. Bu işlem verimliliği artırabilir. Ancak metinlerin uzunlukları farklı olabileceği için girdilerin uygun biçimde düzenlenmesi gerekir.

Padding (dolgu)

Bir batch içindeki dizileri aynı uzunluğa getirmek için kısa dizilerin sonuna özel dolgu token'ları eklenebilir. Böylece girdiler aynı boyutta bir tensör hâline getirilebilir.

Truncation (kırpma)

Modelin kabul edebileceği uzunluk sınırını aşan girdilerin belirli bir uzunlukta kesilmesidir. Çok uzun girdilerde kullanılabilir; ancak kesilen bölümdeki bilgi kaybolabilir.

```
batch = [  
    "I love NLP.",  
    "Transformers help me work with language."  
]  
  
inputs = tokenizer(  
    batch,  
    padding=True,  
    truncation=True,  
    return_tensors="pt"  
)
```

Kod ne yapıyor?

- batch: İki metinden oluşan bir liste.
- padding=True: Batch içindeki dizileri aynı uzunluğa getirir.
- truncation=True: Gerekirse uzun girdileri modelin sınırına göre kısaltır.
- return_tensors="pt": Sonuçları PyTorch tensörleri biçiminde döndürür.

Attention mask

attention_mask, modelin hangi token'ları dikkate alacağını belirtmeye yardımcı olur. Yaygın kullanımda:

- 1: Gerçek girdi token'ı.
- 0: Padding için eklenen token.

Modelin ve tokenizer'ın uygulamasına göre ayrıntılar değişebilir; bu yüzden maskeyi girdinin bir parçası olarak düşünmek en güvenli yaklaşımdır.

6. Bütün adımları birleştirmek

Aşağıdaki örnekte metin önce tokenizer ile hazırlanır, ardından sınıflandırma modeline verilir ve modelin ham skorları alınır.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

text = "This course is useful."
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)

print(outputs.logits)
```

Genel Bakış:

1. Checkpoint belirlenir.
2. Checkpoint'e uygun tokenizer ve görev modeli yüklenir.
3. Metin token ID'lerine dönüştürülür.
4. Girdiler modele gönderilir.
5. Model sınıflandırma için ham skorlar üretir.

Bu örnekte logits yazdırılır; doğrudan “pozitif” veya “negatif” etiketi yazdırılmaz. Etiket görmek için çıktının ayrıca yorumlanması ya da daha üst seviyeli pipeline kullanılması gerekir.

7. Modeli kaydetme ve yeniden yükleme

`save_pretrained()`, modelin veya tokenizer'ın gerekli dosyalarını bir klasöre kaydetmek için kullanılır. Böylece daha sonra aynı kayıt tekrar yüklenebilir.

```
model.save_pretrained("./my_model")
tokenizer.save_pretrained("./my_model")
```

Bu kod modelin ve tokenizer'ın dosyalarını `my_model` klasörüne kaydeder. Daha sonra aynı klasör yolu `from_pretrained()` içinde kullanılarak kayıt yeniden yüklenebilir.

8. Inference (çıkartım) ve dağıtım

Modelin eğitilmiş hâlini tahmin üretmek için kullanmaya **inference (çıkartım)** denir. Modeli bir uygulama veya hizmet üzerinden kullanıma açmak ise **deployment (dağıtım)** olarak adlandırılır.

Hugging Face bölümünde geçen bazı araçlar:

- **TGI (Text Generation Inference):** Metin üretimi modellerini servis etmek için kullanılan araç.
- **vLLM:** Büyük dil modellerinde verimli çıkartım ve servis sunmaya odaklanan araç.
- **llama.cpp:** Özellikle yerel ortamda çalıştırma ve farklı donanımlarda çıkartım yapma senaryolarında kullanılan proje.

Araç seçimi; kullanılacak model, donanım, performans ihtiyacı ve dağıtım ortamına bağlıdır. Bu araçlar aynı şey değildir; farklı kullanım ihtiyaçlarına hitap ederler.

Metin üretiminde karşılaşılabilecek parametreler

- **max_new_tokens:** Üretilcek yeni token sayısına üst sınır koyar.
- **min_new_tokens:** Üretilcek yeni token sayısı için alt sınır belirler.
- **temperature:** Üretimin ne kadar rastlantısal olacağını etkiler. Düşük değerler genellikle daha az çeşitlilik getirir.
- **top_k:** Her adımda değerlendirmeye alınan en yüksek skorlu token seçeneklerini sınırlar.
- **top_p:** Olasılık toplamına göre aday token kümesini sınırlar.
- **Stop sequence (durdurma dizisi):** Üretimin belirli bir metin dizisine ulaştığında durmasını sağlayabilir.
- **Repetition/frequency/presence penalty (tekrar sıklığı/varlığı cezası):** Tekrarları azaltmak veya üretim davranışını yönlendirmek için kullanılan ayarlar.

Bu parametrelerin tümü her araçta aynı adla veya aynı biçimde bulunmayabilir. Kullanılan kütüphanenin ayarları kontrol edilmelidir.